

snickr: Relational Database Design

CS6083 — Project #1

Shichen Zhang (sz4968@nyu.edu)

Zihang Yu (zy3493@nyu.edu)

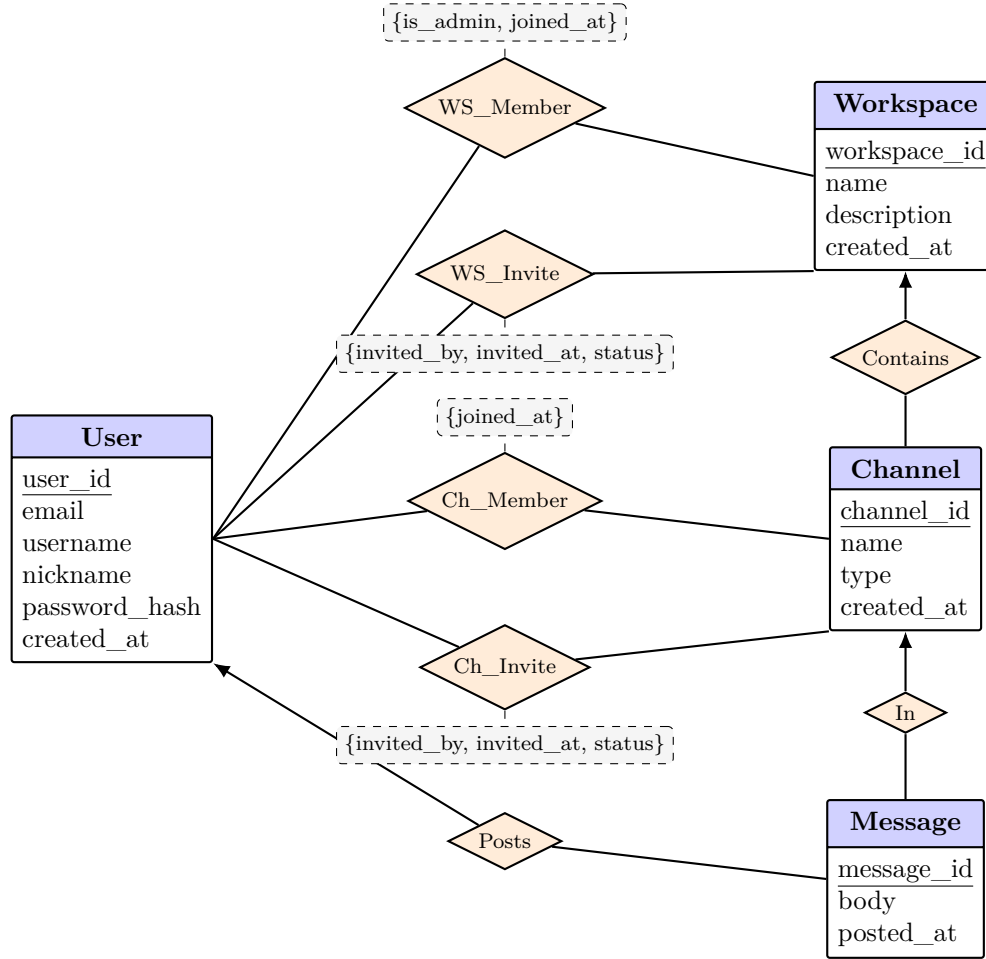
April 27, 2026

Introduction

snickr is a Slack-like collaboration system where users create workspaces, invite members, create public/private/direct channels, and exchange messages inside those channels. This report presents a relational design for the system, including the ER model, relational schema, SQL DDL, required queries, and a small sample dataset used to test the design.

(a) Database Schema Design

ER diagram.



Relational schema.

- **User**(user_id, email, username, nickname, password_hash, created_at)
UNIQUE(email), UNIQUE(username).
- **Workspace**(workspace_id, name, description, created_by, created_at)
FK: created_by → User(user_id); UNIQUE(created_by, name).
- **WorkspaceMember**(workspace_id, user_id, is_admin, joined_at)
FKs: workspace_id → Workspace, user_id → User.
- **WorkspaceInvitation**(workspace_id, invitee_user_id, invited_by, invited_at, status)
FKs: workspace_id → Workspace; invitee_user_id, invited_by → User.
- **Channel**(channel_id, workspace_id, name, type, created_by, created_at)
FKs: workspace_id → Workspace, created_by → User; UNIQUE(workspace_id, name).
- **ChannelMember**(channel_id, user_id, joined_at)
FKs: channel_id → Channel, user_id → User.
- **ChannelInvitation**(channel_id, invitee_user_id, invited_by, invited_at, status)
FKs: channel_id → Channel; invitee_user_id, invited_by → User.
- **Message**(message_id, channel_id, sender_user_id, body, posted_at)
FKs: channel_id → Channel, sender_user_id → User.

Assumptions. Email and username are globally unique; workspace names are unique per creator; channel names are unique only within a workspace; admin is a boolean on workspace membership rather than a separate entity; a direct channel is a channel of type `direct` with exactly two members (cardinality enforced in the application); channel invitations are issued only to users who are already members of the channel’s workspace (enforced in the application); timestamps are stored on every action-bearing row.

Design justification.

- **One Channel table with a type enum** rather than separate tables for public, private, and direct channels. All three share the same attributes, so subtyping would add joins without adding structure.
- **Admin is a boolean on WorkspaceMember**, not a separate entity. Every admin is already a member, so a dedicated **Admin** table would duplicate the membership relationship.
- **Separate membership and invitation tables.** Membership records an accepted state with relationship-level attributes (`is_admin`, `joined_at`); invitation records a pending state with its own attributes (`invited_by`, `invited_at`, `status`). Mixing them in one table would conflate “joined” with “invited but not yet joined”.
- **Separate invitation tables for workspaces and channels.** They point at different parents, so keeping them apart lets each table declare a clean foreign key.
- **Composite natural keys** (`workspace_id`, `user_id`) and (`channel_id`, `user_id`) on the four membership / invitation tables. No surrogate id is needed and the “one row per pair” invariant is structural rather than an extra `UNIQUE` constraint.

Procedures / Application Logic. This design does not rely on database stored procedures or triggers. Multi-step operations are handled by the application using ordinary SQL statements. For example, creating a public channel first inserts the channel and then inserts the creator into `channel_members` if the first step succeeds. Likewise, rules such as “a direct channel has exactly two members” and “channel invitations are issued only to members of the channel’s workspace” are checked by the application before the corresponding insert operations are executed.

(b) SQL Schema (DDL)

```
CREATE TABLE users (
    user_id      BIGSERIAL      PRIMARY KEY,
    email        VARCHAR(255)   NOT NULL UNIQUE,
    username     VARCHAR(64)    NOT NULL UNIQUE,
    nickname     VARCHAR(64)    NOT NULL,
    password_hash VARCHAR(255)  NOT NULL,
    created_at   TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE workspaces (
    workspace_id BIGSERIAL      PRIMARY KEY,
    name         VARCHAR(128)   NOT NULL,
    description   TEXT,
    created_by   BIGINT         NOT NULL REFERENCES users(user_id),
    created_at   TIMESTAMP      NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```

    UNIQUE (created_by, name)
);

CREATE TABLE workspace_members (
    workspace_id BIGINT NOT NULL REFERENCES workspaces(workspace_id),
    user_id BIGINT NOT NULL REFERENCES users(user_id),
    is_admin BOOLEAN NOT NULL DEFAULT FALSE,
    joined_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (workspace_id, user_id)
);

CREATE TABLE workspace_invitations (
    workspace_id BIGINT NOT NULL REFERENCES
        workspaces(workspace_id),
    invitee_user_id BIGINT NOT NULL REFERENCES users(user_id),
    invited_by BIGINT NOT NULL REFERENCES users(user_id),
    invited_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(16) NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending', 'accepted', 'declined')),
    PRIMARY KEY (workspace_id, invitee_user_id)
);

CREATE TABLE channels (
    channel_id BIGSERIAL PRIMARY KEY,
    workspace_id BIGINT NOT NULL REFERENCES
        workspaces(workspace_id),
    name VARCHAR(128) NOT NULL,
    type VARCHAR(16) NOT NULL
        CHECK (type IN ('public', 'private', 'direct')),
    created_by BIGINT NOT NULL REFERENCES users(user_id),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (workspace_id, name)
);

CREATE TABLE channel_members (
    channel_id BIGINT NOT NULL REFERENCES channels(channel_id),
    user_id BIGINT NOT NULL REFERENCES users(user_id),
    joined_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (channel_id, user_id)
);

CREATE TABLE channel_invitations (
    channel_id BIGINT NOT NULL REFERENCES channels(channel_id),
    invitee_user_id BIGINT NOT NULL REFERENCES users(user_id),
    invited_by BIGINT NOT NULL REFERENCES users(user_id),
    invited_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(16) NOT NULL DEFAULT 'pending'
        CHECK (status IN ('pending', 'accepted', 'declined')),
    PRIMARY KEY (channel_id, invitee_user_id)
);

CREATE TABLE messages (
    message_id BIGSERIAL PRIMARY KEY,
    channel_id BIGINT NOT NULL REFERENCES channels(channel_id),

```

```

    sender_user_id BIGINT    NOT NULL REFERENCES users(user_id),
    body           TEXT      NOT NULL,
    posted_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_messages_channel_time ON messages(channel_id,
    posted_at);
CREATE INDEX idx_messages_sender      ON messages(sender_user_id);

```

(c) Required SQL Queries

(1) Create a new user account

```

INSERT INTO users (email, username, nickname, password_hash)
VALUES (:email, :username, :nickname, :password_hash)
RETURNING user_id;

```

(2) Create a public channel (with authorization check)

```

INSERT INTO channels (workspace_id, name, type, created_by)
SELECT :workspace_id, :name, 'public', :user_id
WHERE EXISTS (
    SELECT 1 FROM workspace_members
    WHERE workspace_id = :workspace_id AND user_id = :user_id
)
RETURNING channel_id;

INSERT INTO channel_members (channel_id, user_id)
VALUES (:new_channel_id, :user_id);

```

Here `:new_channel_id` is the `channel_id` returned by the first insert. The second insert is executed by the application only if the first insert returns a `channel_id`.

(3) For each workspace, list all administrators

```

SELECT w.workspace_id, w.name AS workspace, u.username AS administrator
FROM   workspaces w
JOIN   workspace_members wm ON wm.workspace_id = w.workspace_id
JOIN   users u             ON u.user_id       = wm.user_id
WHERE  wm.is_admin = TRUE
ORDER BY w.workspace_id, u.username;

```

(4) Per public channel: # invitees invited >5 days ago, not yet joined

```

SELECT c.channel_id, c.name,
    COUNT(ci.invitee_user_id) FILTER (WHERE cm.user_id IS NULL)
    AS stale_unjoined_invites
FROM   channels c
LEFT   JOIN channel_invitations ci
    ON ci.channel_id = c.channel_id

```

```

        AND ci.invited_at < CURRENT_TIMESTAMP - INTERVAL '5 days'
LEFT JOIN channel_members cm
      ON cm.channel_id = ci.channel_id AND cm.user_id =
         ci.invitee_user_id
WHERE  c.workspace_id = :workspace_id
      AND c.type = 'public'
GROUP BY c.channel_id, c.name
ORDER BY c.name;

```

(5) All messages in a channel, chronological order

```

SELECT m.message_id, m.posted_at, u.username, m.body
FROM   messages m
JOIN   users u ON u.user_id = m.sender_user_id
WHERE  m.channel_id = :channel_id
ORDER BY m.posted_at ASC, m.message_id ASC;

```

(6) All messages posted by a given user

```

SELECT m.message_id, w.name AS workspace, c.name AS channel,
       m.posted_at, m.body
FROM   messages m
JOIN   channels c ON c.channel_id = m.channel_id
JOIN   workspaces w ON w.workspace_id = c.workspace_id
WHERE  m.sender_user_id = :user_id
ORDER BY m.posted_at DESC;

```

(7) Accessible messages containing “perpendicular”

```

SELECT m.message_id, w.name AS workspace, c.name AS channel,
       u.username AS author, m.posted_at, m.body
FROM   messages m
JOIN   channels c ON c.channel_id = m.channel_id
JOIN   workspaces w ON w.workspace_id = c.workspace_id
JOIN   users u ON u.user_id = m.sender_user_id
JOIN   channel_members cm ON cm.channel_id = c.channel_id
                        AND cm.user_id = :user_id
JOIN   workspace_members wm ON wm.workspace_id = w.workspace_id
                        AND wm.user_id = :user_id
WHERE  m.body ILIKE '%perpendicular%'
ORDER BY m.posted_at DESC;

```

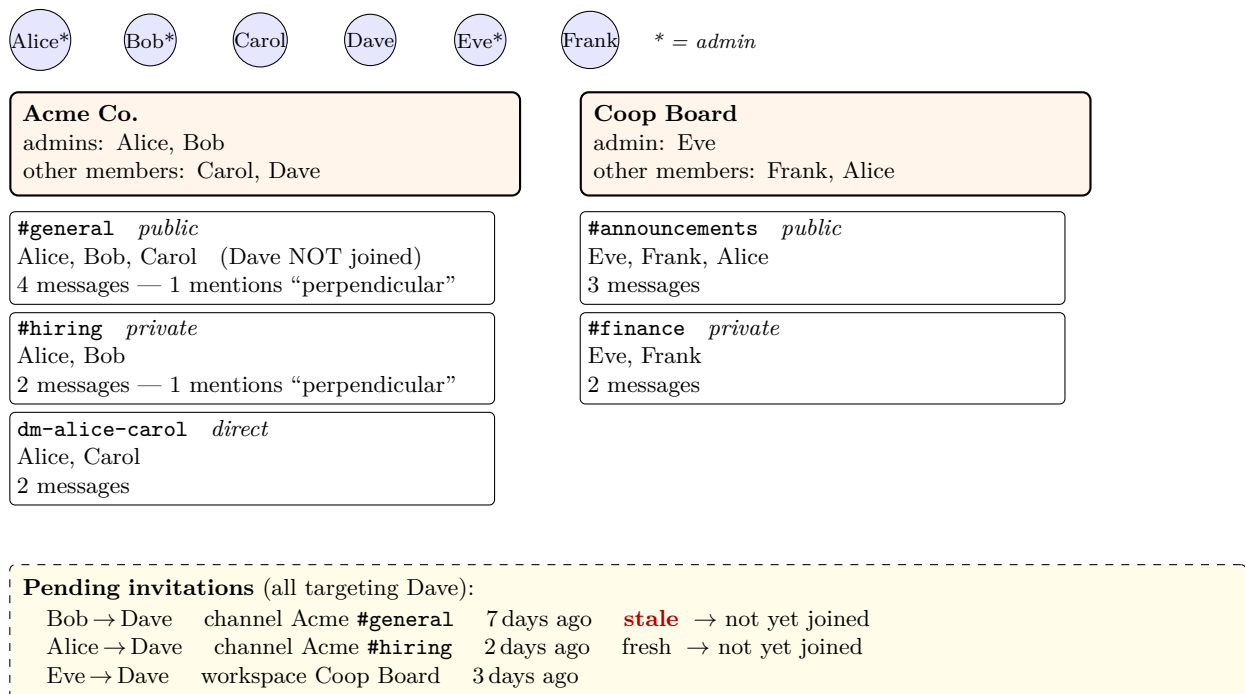
(d) Sample Data and Testing

Sample Data

The sample data populates 6 users, 2 workspaces, 5 channels, 13 messages, and 3 pending invitations. The diagram below summarizes the data as a small workspace map. Alice, Bob, Carol, Dave, Eve, and Frank are the users; asterisks mark users who are administrators in at least one workspace. The

first workspace, Acme Co., has Alice and Bob as admins and Carol and Dave as regular members. It contains a public **#general** channel, a private **#hiring** channel, and a direct channel between Alice and Carol. Dave is listed as an Acme workspace member but is intentionally not a member of any Acme channel.

The second workspace, Coop Board, has Eve as admin and Frank and Alice as regular members, so Alice appears in both workspaces. Coop Board contains a public **#announcements** channel and a private **#finance** channel. The message counts are shown inside each channel box, and channels include notes about whether any message contains the word “perpendicular”. The dashed invitation box records three still-pending invitations to Dave: one stale channel invitation in Acme, one recent channel invitation in Acme, and one workspace invitation in Coop Board.



Query Testing

After loading the sample data, the queries in part (c) were executed against the populated database. The observed results are shown below.

(1) Create a new user account

This test inserts a new user named Grace and returns the generated `user_id`.

Query.

```
INSERT INTO users (email, username, nickname, password_hash)
VALUES ('grace@acme.com', 'grace', 'Grace', 'hash_grace')
RETURNING user_id;
```

Result.

```
user_id
-----
7
(1 row)

INSERT 0 1
```

(2) Create a public channel (with authorization check)

This test first lets Alice (`user_id=1`), who is a member of Acme (`workspace_id=1`), create a new public channel `#random`; it then adds her to `channel_members`. The second test lets Eve (`user_id=5`) attempt the same operation in Acme (`workspace_id=1`), but since she is not a member of that workspace, the guarded insert returns no rows.

Query.

```
-- Test A: Alice, who is an Acme member, creates #random.
INSERT INTO channels (workspace_id, name, type, created_by)
SELECT 1, '#random', 'public', 1
WHERE EXISTS (
    SELECT 1 FROM workspace_members
    WHERE workspace_id = 1 AND user_id = 1
)
RETURNING channel_id;

INSERT INTO channel_members (channel_id, user_id)
VALUES (6, 1);

-- Test B: Eve tries to create a channel in Acme, but she is not a
-- member.
INSERT INTO channels (workspace_id, name, type, created_by)
SELECT 1, '#eve-unauthorized', 'public', 5
WHERE EXISTS (
    SELECT 1 FROM workspace_members
    WHERE workspace_id = 1 AND user_id = 5
)
RETURNING channel_id;
```

Result.

Test A: Alice, who is an Acme member, creates `#random`.

```
channel_id
-----
6
(1 row)

INSERT 0 1
INSERT 0 1
```

Test B: Eve tries to create a channel in Acme, but she is not a member.

```
channel_id
-----
(0 rows)

INSERT 0 0
```

(3) For each workspace, list all administrators

This query joins `workspaces`, `workspace_members`, and `users` to list every administrator in each workspace.

Query.

```
SELECT w.workspace_id, w.name AS workspace, u.username AS administrator
FROM   workspaces w
JOIN   workspace_members wm ON wm.workspace_id = w.workspace_id
JOIN   users u              ON u.user_id       = wm.user_id
WHERE  wm.is_admin = TRUE
ORDER BY w.workspace_id, u.username;
```

Result.

workspace_id	workspace	administrator
1	Acme Co.	alice
1	Acme Co.	bob
2	Coop Board	eve

(3 rows)

(4) Per public channel: # invitees invited >5 days ago, not yet joined

This query counts, for each public channel in Acme (`workspace_id=1`), the pending invitations that are older than five days and whose invitees still do not appear in `channel_members`.

Query.

```
SELECT c.channel_id, c.name,
       COUNT(ci.invitee_user_id) FILTER (WHERE cm.user_id IS NULL)
       AS stale_unjoined_invites
FROM   channels c
LEFT JOIN channel_invitations ci
      ON ci.channel_id = c.channel_id
      AND ci.invited_at < CURRENT_TIMESTAMP - INTERVAL '5 days'
LEFT JOIN channel_members cm
      ON cm.channel_id = ci.channel_id AND cm.user_id =
         ci.invitee_user_id
WHERE  c.workspace_id = 1
      AND c.type = 'public'
GROUP BY c.channel_id, c.name
ORDER BY c.name;
```

Result.

channel_id	name	stale_unjoined_invites
1	#general	1
6	#random	0

(2 rows)

(5) All messages in a channel, chronological order

This test retrieves all messages in Acme #general (`channel_id=1`) and orders them by `posted_at` and then by `message_id`.

Query.

```
SELECT m.message_id, m.posted_at, u.username, m.body
FROM   messages m
JOIN   users u ON u.user_id = m.sender_user_id
WHERE  m.channel_id = 1
ORDER BY m.posted_at ASC, m.message_id ASC;
```

Result.

message_id	posted_at	username	body
1	2026-04-14 21:26:56.771839	alice	Welcome to Acme #general!
2	2026-04-16 21:26:56.771839	bob	Standup notes will be posted here daily.
3	2026-04-18 21:26:56.771839	carol	Quick geometry question: are these lines perpendicular?
4	2026-04-19 21:26:56.771839	alice	Let's meet tomorrow at 10am.

(4 rows)

(6) All messages posted by a given user

This query lists all messages posted by Alice (`user_id=1`) across all channels and workspaces, ordered from newest to oldest.

Query.

```

SELECT m.message_id, w.name AS workspace, c.name AS channel,
       m.posted_at, m.body
FROM   messages m
JOIN   channels    c ON c.channel_id    = m.channel_id
JOIN   workspaces  w ON w.workspace_id  = c.workspace_id
WHERE  m.sender_user_id = 1
ORDER BY m.posted_at DESC;

```

Result.

message_id	workspace	channel	posted_at	body
4	Acme Co.	#general	2026-04-19 21:26:56.771839	Let's meet tomorrow at 10am.
11	Coop Board	#announcements	2026-04-17 21:26:56.771839	Could someone share last month's minutes?
5	Acme Co.	#hiring	2026-04-15 21:26:56.771839	Candidate feedback for the backend role.
1	Acme Co.	#general	2026-04-14 21:26:56.771839	Welcome to Acme #general!
7	Acme Co.	dm-alice-carol	2026-04-11 21:26:56.771839	Hey Carol, got a minute?

(5 rows)

(7) Accessible messages containing “perpendicular”

This test checks accessibility for three different users: Alice (`user_id=1`), Carol (`user_id=3`), and Dave (`user_id=4`). Alice is a member of both Acme channels that contain the keyword, Carol is only a member of `#general`, and Dave is an Acme workspace member but not a member of any Acme channel.

Query.

```

-- Test A: Alice (user_id = 1)
SELECT m.message_id, w.name AS workspace, c.name AS channel,
       u.username AS author, m.posted_at, m.body
FROM   messages m
JOIN   channels    c ON c.channel_id    = m.channel_id
JOIN   workspaces  w ON w.workspace_id  = c.workspace_id
JOIN   users       u ON u.user_id       = m.sender_user_id
JOIN   channel_members cm ON cm.channel_id = c.channel_id
                        AND cm.user_id    = 1
JOIN   workspace_members wm ON wm.workspace_id = w.workspace_id
                        AND wm.user_id    = 1
WHERE  m.body ILIKE '%perpendicular%'
ORDER BY m.posted_at DESC;

-- Test B: Carol (user_id = 3)
SELECT m.message_id, w.name AS workspace, c.name AS channel,
       u.username AS author, m.posted_at, m.body
FROM   messages m
JOIN   channels    c ON c.channel_id    = m.channel_id
JOIN   workspaces  w ON w.workspace_id  = c.workspace_id
JOIN   users       u ON u.user_id       = m.sender_user_id
JOIN   channel_members cm ON cm.channel_id = c.channel_id
                        AND cm.user_id    = 3
JOIN   workspace_members wm ON wm.workspace_id = w.workspace_id
                        AND wm.user_id    = 3
WHERE  m.body ILIKE '%perpendicular%'
ORDER BY m.posted_at DESC;

-- Test C: Dave (user_id = 4)
SELECT m.message_id, w.name AS workspace, c.name AS channel,
       u.username AS author, m.posted_at, m.body

```

```

FROM    messages m
JOIN    channels      c  ON c.channel_id    = m.channel_id
JOIN    workspaces    w  ON w.workspace_id  = c.workspace_id
JOIN    users          u  ON u.user_id      = m.sender_user_id
JOIN    channel_members cm ON cm.channel_id  = c.channel_id
                        AND cm.user_id      = 4
JOIN    workspace_members wm ON wm.workspace_id = w.workspace_id
                        AND wm.user_id      = 4
WHERE   m.body ILIKE '%perpendicular%'
ORDER BY m.posted_at DESC;

```

Result.**Test A: Alice (user_id = 1).**

message_id	workspace	channel	author	posted_at	body
3	Acme Co.	#general	carol	2026-04-18 21:26:56.771839	Quick geometry question: are these lines perpendicular?
6	Acme Co.	#hiring	bob	2026-04-17 21:26:56.771839	Their system-design answer had two perpendicular approaches.

(2 rows)

Test B: Carol (user_id = 3).

message_id	workspace	channel	author	posted_at	body
3	Acme Co.	#general	carol	2026-04-18 21:26:56.771839	Quick geometry question: are these lines perpendicular?

(1 row)

Test C: Dave (user_id = 4).

message_id	workspace	channel	author	posted_at	body
------------	-----------	---------	--------	-----------	------

(0 rows)